

Navigators IDR - Implementation Plans & Progress Report

Project: Navigators (SIH26168 for ISRO)

Objective: AI/ML-powered navigation system demonstrating seamless vehicle tracking during GNSS denial using Intelligent Dead Reckoning.

Phase 1: Core Navigation Engine & AI Model (Completed)

Plan

1. **Data Pipeline:** Build loaders for the VNBD dataset (IMU & GNSS data) to prepare for model training.
2. **AI Model:** Implement a Temporal Convolutional Network (TCN) in PyTorch to estimate velocity from IMU (accelerometer & gyroscope) data.
3. **Navigation Engine (EKF):** Develop an Extended Kalman Filter (15-state) to fuse GNSS and IMU data.
4. **Dead Reckoning (DR) Engine:** Implement a system that takes over when GNSS is denied, integrating the AI-estimated velocity to update position.
5. **Constraints:** Implement Non-Holonomic Constraints (NHC) and Zero Velocity Update (ZUPT) as pseudo-measurements to limit drift.

What We Did

- Created `src/data/io_vnbd_loader.py` and `src/data/dataset.py`.
 - Developed `src/models/tcn_model.py` and training scripts (`scripts/train.py`, `scripts/evaluate.py`).
 - Built the navigation logic in `src/navigation/` (`ekf.py`, `dead_reckoning.py`, `nhc.py`, `zupt.py`).
 - Wrote extensive unit tests (51/51 passing) verifying components like Haversine distance, EKF updates, and ZUPT logic.
-

Phase 2: Web Simulator & Model Export (Completed)

Plan

1. **Simulation UI:** Create a visual dashboard to demonstrate the system working.
2. **Visualization:** Use Leaflet.js to plot Ground Truth vs. Estimated Trajectory on a map.
3. **Telemetry:** Add gauges for Speed, Heading, Position Error, and Drift Percentage.
4. **Deployment Prep:** Create an export script to convert the PyTorch model to ONNX format for edge deployment.

What We Did

- Built a web-based simulator (`simulator/index.html`, `index.css`, `app.js`).
- Integrated playback controls to simulate historical data runs.
- Created `src/edge/onnx_export.py` and successfully exported the PyTorch model.
- Demonstrated automatic mode switching (GNSS -> Dead Reckoning) with visual indicators in the simulator.

Phase 3: Live Mobile Sensor Integration (Completed)

Plan

1. **Backend Server:** Build a FastAPI server (`src/api/server.py`) to receive live data via WebSockets.
2. **Mobile Frontend:** Upgrade the simulator to a Progressive Web App (PWA) with a Service Worker (`simulator/manifest.json`, `simulator/sw.js`).
3. **Sensor Capture:** Write `live_sensor.js` to capture live `DeviceMotionEvent` (Accelerometer, Gyroscope) and Geolocation (GNSS) from a mobile phone and stream it to the backend.

What We Did

- Successfully wired up the mobile sensors to the Python backend.
 - Deployed a WebSocket pipeline for real-time EKF processing on the Mac, with the mobile device acting as a thin client/sensor node.
-

Phase 4: 100% Offline Edge Processing (Completed)

Plan

(Pivoted architecture to meet ISRO's strict GNSS-denied/Offline requirement)

1. **Edge AI:** Discard the Python backend dependency for live runs.
2. **WebAssembly Integration:** Embed ONNX Runtime Web (`ort.min.js`) into the PWA.
3. **Local Navigation Engine:** Write a Javascript-based Edge Engine (`offline_engine.js`) to process sensor buffers, run the ONNX AI model, and perform Dead Reckoning directly on the mobile device CPU.
4. **Offline Caching:** Update the Service Worker to cache the AI model (`model.onnx`) and all assets for Airplane Mode execution.

What We Did

- Installed `onnx` and `onnxscript` and exported the TCN model to `simulator/model.onnx`.
 - Completely refactored the web app to run autonomously without any server.
 - The system now functions entirely on the edge (mobile phone), capturing data, running the neural network, and plotting the trajectory even when all networks (Wi-Fi, Cellular) are disabled.
-

Phase 5: Hardened Sensor Pipeline & Edge EKF (Completed)

Plan

1. **Sensor Pipeline:** Build a robust preprocessor with non-linear median filtering to remove pothole/shock vibrations from IMU data.
2. **ML Augmentations:** Introduce continuous Gaussian noise, impulsive noise, speed variation, and random heading misalignment during AI model training to guarantee generalization.
3. **Edge Engine Rewrite:** Port the entire Python EKF (15-state), Map Matching, ZUPT, and NHC logic to purely functional JavaScript for execution inside the PWA without external backends.

What We Did

- Developed matrix.js and ekf.js to run a fully optimized 15-state Extended Kalman Filter natively in the browser.
 - Ported Geometric Map Matching to map_matcher.js for offline snapping to road networks.
 - Upgraded src/models/trainer.py with rigorous batch augmentations.
 - Upgraded src/data/preprocessor.py with median filtering to protect the velocity inference from violent mechanical shocks.
-

Current Position & Readiness

Current State: System Ready for Field Testing

We have successfully built a full-stack, AI-powered Intelligent Dead Reckoning system that meets the extreme operational constraints of the ISRO problem statement.

Key Capabilities Currently Active:

- **Zero-Connectivity Operations:** The entire application is a self-contained PWA that installs on the device and operates in Airplane mode.
- **Edge Inference:** The PyTorch model is running in WebAssembly on the mobile CPU.
- **Full Edge EKF:** A 15-state Extended Kalman Filter, complete with ZUPT, NHC, and Map Matching runs natively in JS.
- **Seamless Handover:** The system initializes with GNSS (if available) and seamlessly falls back to AI-driven Dead Reckoning when GNSS is lost.

Next Steps (Optional/Future Work):

- **Field Data Collection:** Collect real-world IMU data on specific vehicles to fine-tune the ONNX model.
- **Native Android App:** If browser sensor limitations become a bottleneck, port the WebAssembly solution to a native Android APK using React Native or Flutter.